

Solidity y contratos inteligentes

Informe de laboratorio. Guía n.º 3

Rodríguez Ángel, Santiago

Olaya Rojas, Santiago

Universidad Manuela Beltrán

Ingeniería de Software

Facultad de Ingeniería

Nuevas Tecnologías del Desarrollo

Grupo 6

Bogotá, Colombia

2026

Tabla de contenido

Introducción 5
1. Marco conceptual 6
1.1 Contrato inteligente 6
1.2 Solidity y el contrato de ejemplo 6
1.3 Transacciones, llamadas y gas 7
1.4 Lectura previa: el caso Esbrinachain 7
2. Desarrollo del laboratorio 9
2.1 Entorno y método de ejecución 9
2.2 Paso 1. Contrato inteligente universitario: EntregaEvidencias 9
2.3 Paso 2. Crear el contrato en Remix 12
2.4 Paso 3. Compilar el contrato 12
2.5 Paso 4. Despliegue y pruebas del contrato original 13
2.6 Actividad: agregar precio y establecerPrecio 15
2.7 Reflexión: por qué escribir cuesta gas y leer no 17
Conclusiones 19
Referencias 21
Anexo A. Contrato original (MiContrato.sol) 22
Anexo B. Contrato modificado (MiContrato_v2.sol) 23

Lista de tablas

Tabla 1 Datos de la práctica 5
Tabla 2 Dónde está cada elemento del entregable 6
Tabla 3 Elementos del contrato MiPrimerContrato 7
Tabla 4 Del contrato de la guía al caso Esbrinachain 8
Tabla 5 Entorno de ejecución del laboratorio 9
Tabla 6 Partes y responsabilidades del contrato EntregaEvidencias 10
Tabla 7 Condiciones y acciones que ejecuta el contrato 10
Tabla 8 Funciones propuestas del contrato EntregaEvidencias 11
Tabla 9 Pruebas del contrato original en Remix VM (Cancun) 14
Tabla 10 Descomposición del costo de la transacción cambiarMensaje 19

Lista de figuras

Figura 1 Estados de una entrega en el contrato EntregaEvidencias 11
Figura 2 Compilación de MiContrato.sol sin errores 13
Figura 3 Contrato original funcionando: mensaje y contador 14
Figura 4 Transacciones registradas en la consola de Remix 15
Figura 5 Contrato modificado compilado sin errores 16
Figura 6 Contrato modificado desplegado y funcionando 17
Figura 7 Costo de una escritura frente a una lectura en Remix 18

Introducción

Este informe documenta la guía de laboratorio n.º 3, «Solidity y contratos», del módulo de aplicaciones multiplataforma de Nuevas Tecnologías del Desarrollo. El objetivo de la guía es comprender qué es un contrato inteligente, conocer el lenguaje Solidity, y escribir, desplegar y probar un contrato básico con Remix IDE (Universidad Manuela Beltrán [UMB], 2026).

El trabajo se hizo sobre el contrato de ejemplo de la guía, que guarda un mensaje, un propietario y un contador. Se compiló, se desplegó en una cadena simulada y se probó. Después se modificó para agregar un precio, y se midió con cifras reales cuánto cuesta una escritura frente a una lectura, para responder la pregunta de reflexión. También se diseñó un contrato para un caso del entorno universitario, apoyado en la lectura previa sobre el proyecto Esbrinachain del libro *Blockchain práctico*.

Todas las capturas son de ejecuciones reales de Remix; en la sección 2.1 se explica el entorno y cómo se operó. La Tabla 1 reúne los datos de la práctica y la Tabla 2 indica dónde está cada elemento del entregable.

Tabla 1
Datos de la práctica

Campo	Valor
Grupo	Grupo 6
Integrante 1	Rodríguez Ángel, Santiago
Integrante 2	Olaya Rojas, Santiago
Fecha de ejecución	14 de octubre de 2026
Asignatura	Nuevas Tecnologías del Desarrollo, módulo 1: Aplicaciones multiplataforma
Guía	Guía de laboratorio n.º 3: Solidity y contratos (Smart Contracts, lenguaje Solidity, Remix IDE)

Nota. Elaboración propia con base en la guía (UMB, 2026).

Tabla 2
Dónde está cada elemento del entregable

Elemento del entregable	Dónde está
1. Captura de Remix con el contrato original funcionando (mensaje y contador)	Figura 3 y Figura 4, sección 2.5
2. Código del contrato modificado con la variable precio (.sol)	Sección 2.6, Anexo B y archivo contratos/MiContrato_v2.sol
3. Captura del contrato modificado desplegado y funcionando	Figura 5 y Figura 6, sección 2.6
4. Respuesta escrita a la reflexión del Paso 4	Sección 2.7
5. Diseño del contrato inteligente universitario del Paso 1	Sección 2.2, Figura 1 y tablas de la sección

Nota. Elaboración propia con base en el entregable de la guía (UMB, 2026).

1. Marco conceptual

1.1 Contrato inteligente

Un contrato inteligente es un programa que se ejecuta en la blockchain de Ethereum: un conjunto de código (sus funciones) y datos (su estado) que reside en una dirección específica (ethereum.org, 2026b). Las reglas quedan escritas en el código y se aplican solas cuando se cumplen las condiciones, sin intermediarios; además, cualquiera puede leer el código y, una vez desplegado, no se modifica (UMB, 2026). Por defecto no se pueden eliminar y sus interacciones son irreversibles (ethereum.org, 2026b).

Tienen un límite importante: por sí solos no pueden obtener información de eventos del mundo real ni recuperar datos de fuentes externas a la cadena. Para eso se usan oráculos, herramientas que llevan datos externos al contrato (ethereum.org, 2026b). Esa limitación condiciona el diseño del caso universitario de la sección 2.2.

1.2 Solidity y el contrato de ejemplo

Solidity es el lenguaje con el que se escriben contratos para la máquina virtual de Ethereum. La Tabla 3 explica, elemento por elemento, el contrato *MiPrimerContrato* de la guía, cuyo texto completo está en el Anexo A.

Tabla 3

Elementos del contrato MiPrimerContrato

Elemento	Qué hace
// SPDX-License-Identifier: MIT	Indica la licencia del código fuente (MIT).
pragma solidity ^0.8.0;	Versión del compilador admitida: desde la 0.8.0 hasta antes de la 0.9.0. Remix usó la 0.8.34.
contract MiPrimerContrato	Agrupa código (funciones) y datos (estado) que residen en una dirección de la cadena (Solidity, s. f.-b).
string public mensaje; ***address public propietario;*** ***uint public contador;***	Variables de estado: viven en el almacenamiento (*storage*), que persiste entre llamadas y transacciones (Solidity, s. f.-b). Al ser *public*, el compilador crea una función de lectura para cada una (Solidity, s. f.-a). *uint* es un alias de *uint256* y *address* guarda un valor de 20 bytes (Solidity, s. f.-c).
constructor(string memory _mensajeInicial)	Se ejecuta una sola vez, cuando se crea el contrato (Solidity, s. f.-a). *memory* es la ubicación de datos que Solidity exige para tipos por referencia como *string* (Solidity, s. f.-c).
propietario = msg.sender;	*msg.sender* es la dirección de la que viene la llamada externa actual (Solidity, s. f.-b); en el constructor es la cuenta que despliega el contrato.
cambiarMensaje(...)* y *contador++	Escribe en el almacenamiento. Es una transacción: la envía una cuenta, la ejecutan los nodos y cuesta gas.
obtenerContador() ... view	*view* promete no modificar el estado (Solidity, s. f.-a); por eso solo lee.

Nota. Elaboración propia con base en la guía (UMB, 2026) y en la documentación de Solidity (s. f.-a, s. f.-b, s. f.-c).

1.3 Transacciones, llamadas y gas

Una transacción es un mensaje que una cuenta envía a otra; se aplica completa o no se aplica (Solidity, s. f.-b). Cada transacción debe pagarse con gas, que mide el esfuerzo computacional de una operación, y lo paga quien origina la transacción. El gas existe para que la red no sea vulnerable al spam ni se quede atrapada en ciclos de cómputo infinitos (ethereum.org, 2026a; Solidity, s. f.-b). El costo total es el gas usado multiplicado por el precio por unidad, que suma la tarifa base y la propina al validador (ethereum.org, 2026a).

El almacenamiento es la parte más costosa de la máquina virtual: es comparativamente caro leerlo y más aún inicializarlo y modificarlo (Solidity, s. f.-b). Esta idea es la base de la reflexión de la sección 2.7.

1.4 Lectura previa: el caso Esbrinachain

La lectura sobre contratos inteligentes del libro *Blockchain práctico* (sección 3.1) presenta Esbrinachain, un proyecto cuyos contratos ponen por escrito las reglas del sistema: quién

pregunta, quién responde y quién vota, y cuándo. Sirven como ejemplo de cómo el contrato de la guía crece hasta un caso real (Tabla 4).

Tabla 4
Del contrato de la guía al caso Esbrinachain

En el contrato de la guía	En Esbrinachain
propietario = msg.sender	Un administrador (*esbAdmin*) y el modificador *soloAdmin*, que reserva funciones para él.
Variables de estado como *contador*	Contadores (*total_preg*, *total_usuarios*) y mapeos de preguntas y usuarios.
***cambiarMensaje* (escritura)**	*creaPregunta*, *creaRespuesta* y *votarRespuesta*, que validan plazos, estados y votos únicos.
***obtenerContador* (*view*)**	*estadoPreg*, que devuelve el estado de una pregunta sin gastar gas.
(no aparece en la guía)	Funciones *payable*: la recompensa de cada pregunta queda en el saldo del contrato.
Un solo contrato	Tres contratos con herencia: preguntar, responder y el principal, que hereda de los dos.

Nota. Elaboración propia con base en la lectura del libro *Blockchain práctico*, sección 3.1.

Dos ideas de esa lectura se aprovechan en el diseño de la sección 2.2. La primera es que los plazos se comprueban cuando alguien llama al contrato: el contrato no tiene un reloj que lo despierte, y por ejemplo la pregunta pasa a votación cuando llega la primera respuesta fuera de plazo. La segunda es que el tamaño del contrato y el gas obligan a ser austero: cada contrato tiene un límite de tamaño, los ciclos deben ser cortos y finitos, y conviene acortar mensajes de error y evitar variables innecesarias.

2. Desarrollo del laboratorio

2.1 Entorno y método de ejecución

El laboratorio se ejecutó en Remix IDE, el entorno en línea que pide la guía. La dirección remix.ethereum.org redirigió a app.remix.live, y la propia aplicación mostró el aviso «Welcome to the new home of Remix». Las acciones sobre la interfaz (crear el archivo, pegar el código, compilar, desplegar y llamar las funciones) las hizo un script que controla Microsoft Edge mediante el protocolo Chrome DevTools, y las capturas son recortes de esa ejecución real, sin retoques. En el cuadro de preferencias de Remix se desactivaron la analítica de rendimiento y el asistente de inteligencia artificial; la analítica anónima que Remix marca como necesaria no se puede desactivar. La Tabla 5 resume el entorno.

Tabla 5
Entorno de ejecución del laboratorio

Parámetro	Valor
Fecha de ejecución	14 de octubre de 2026
IDE	Remix IDE 2.6.6 (app.remix.live)
Compilador de Solidity	0.8.34+commit.80d5c536
Entorno de ejecución	Remix VM (Cancun), una cadena simulada
Cuenta de prueba	Account 1, 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4, con 100 ETH simulados
Navegador	Microsoft Edge 155, controlado por script (herramientas/cdp.mjs)

Nota. Elaboración propia con datos de la ejecución en *Remix IDE* (s. f.).

La guía pide «Remix VM (Cancun)». Esta versión de Remix trae «Osaka» como versión de la EVM por defecto, así que se seleccionó Cancun en el menú desplegable para seguir el procedimiento tal como está escrito.

2.2 Paso 1. Contrato inteligente universitario: EntregaEvidencias

Actividad. La guía pide diseñar un contrato inteligente para un caso real del entorno universitario, definiendo las condiciones, las partes y qué se ejecuta automáticamente. Se desarrolló el ejemplo de la guía (registrar la nota 5,0 a quien sube todas las evidencias antes de la

fecha límite) hasta un diseño con reglas completas, a partir de lo aprendido en la lectura de Esbrinachain.

Partes. Intervienen cuatro actores (Tabla 6). El docente cumple el papel del administrador de Esbrinachain y el estudiante, el del usuario que actúa sobre sus propios datos.

Tabla 6
Partes y responsabilidades del contrato EntregaEvidencias

Parte	Rol	Qué puede hacer
Docente	Administrador del contrato	Configura la fecha límite, el número de evidencias requeridas y el plazo de revisión; anula una entrega, con motivo, dentro del plazo de revisión.
Estudiante	Usuario	Registra la huella (*hash*) de cada evidencia, una sola vez, y consulta su estado y su nota.
Contrato	Árbitro automático	Verifica la fecha y la cantidad, cambia el estado, registra la nota y emite eventos.
Coordinación académica	Observador	Consulta el estado de cualquier entrega: la lectura es pública y no gasta gas.

Nota. Elaboración propia.

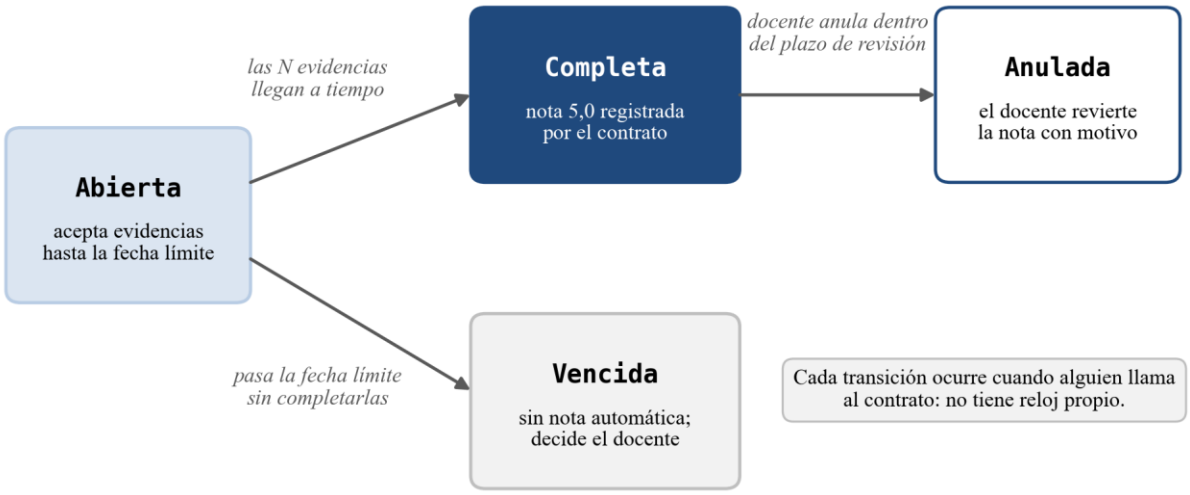
Condiciones y ejecución automática. La Tabla 7 enumera las condiciones y lo que el contrato ejecuta solo en cada caso, y la Figura 1 muestra los estados por los que pasa una entrega.

Tabla 7
Condiciones y acciones que ejecuta el contrato

Condición	Qué ejecuta el contrato
1. El docente fija la fecha límite, las N evidencias requeridas y el plazo de revisión.	Guarda los parámetros. Solo el docente puede (modificador *soloDocente*).
2. El estudiante registra una evidencia antes de la fecha límite y su huella no estaba registrada.	Guarda la huella y el momento del bloque, suma una evidencia y emite *EvidenciaRegistrada*.
3. El número de evidencias llega a N antes de la fecha límite.	Pasa la entrega a *Completa*, registra la nota 5,0 y emite *NotaRegistrada*.
4. Pasa la fecha límite sin llegar a N.	La entrega se evalúa como *Vencida* cuando alguien la consulta o intenta registrar; no se registra nota automática.
5. El docente detecta un problema dentro del plazo de revisión.	Pasa la entrega a *Anulada*, deja la nota en 0 y emite el motivo en *EntregaAnulada*.
6. Alguien intenta algo fuera de las reglas (evidencia repetida, fecha vencida, no ser el docente).	Revierte la transacción y no cambia nada.

Nota. Elaboración propia.

Figura 1
Estados de una entrega en el contrato EntregaEvidencias



Nota. Elaboración propia. N es el número de evidencias requeridas que fija el docente.

Funciones propuestas. La Tabla 8 resume la interfaz del contrato. La nota se guardaría como un entero en décimas (5,0 se representa como 50), porque en Solidity se trabaja con enteros.

Tabla 8
Funciones propuestas del contrato EntregaEvidencias

Función	Quién	Tipo	Efecto
configurar(fechaLimite, requeridas, plazoRevision)	Docente	Escritura	Guarda los parámetros antes de abrir la entrega.
registrarEvidencia(bytes32 huella)	Estudiante	Escritura (paga gas)	Registra una evidencia y, si completa las N a tiempo, registra la nota.
anularEntrega(address estudiante, string motivo)	Docente	Escritura	Anula una entrega completa dentro del plazo de revisión.
estadoEntrega(address estudiante)	Cualquiera	*view* (sin gas)	Devuelve el estado, la cantidad de evidencias y la nota.

Nota. Elaboración propia. Es una interfaz de diseño: no se implementó ni se desplegó en este laboratorio.

Decisiones de diseño. Seis decisiones sostienen la coherencia del contrato:

Huella en lugar de archivo. El contrato no puede abrir un archivo ni consultar el aula virtual, porque los contratos no recuperan datos externos a la cadena (ethereum.org, 2026b). Por eso solo guarda la huella de cada evidencia, que sirve de prueba de que existía en ese momento sin exponer su contenido.

Tiempo del bloque. La fecha de cada registro la da la cadena, no el reloj del estudiante, lo que evita discusiones sobre cuándo se entregó.

Plazos evaluados al llamar. Como en Esbrinachain, el contrato no tiene reloj propio: el estado *Vencida* se determina cuando alguien consulta o intenta registrar.

Una nota que certifica entrega, no calidad. El 5,0 automático significa «entregó todo a tiempo»; la calificación académica sigue siendo decisión del docente, y por eso existen el plazo de revisión y la anulación.

Sin datos personales en la cadena. Lo que se escribe no se puede borrar (ethereum.org, 2026b), de modo que se guardan direcciones y huellas, no nombres.

Costo del gas. Cada registro es una transacción que cuesta gas (sección 2.7); en un uso real convendría una red de pruebas o una red privada de la universidad.

2.3 Paso 2. Crear el contrato en Remix

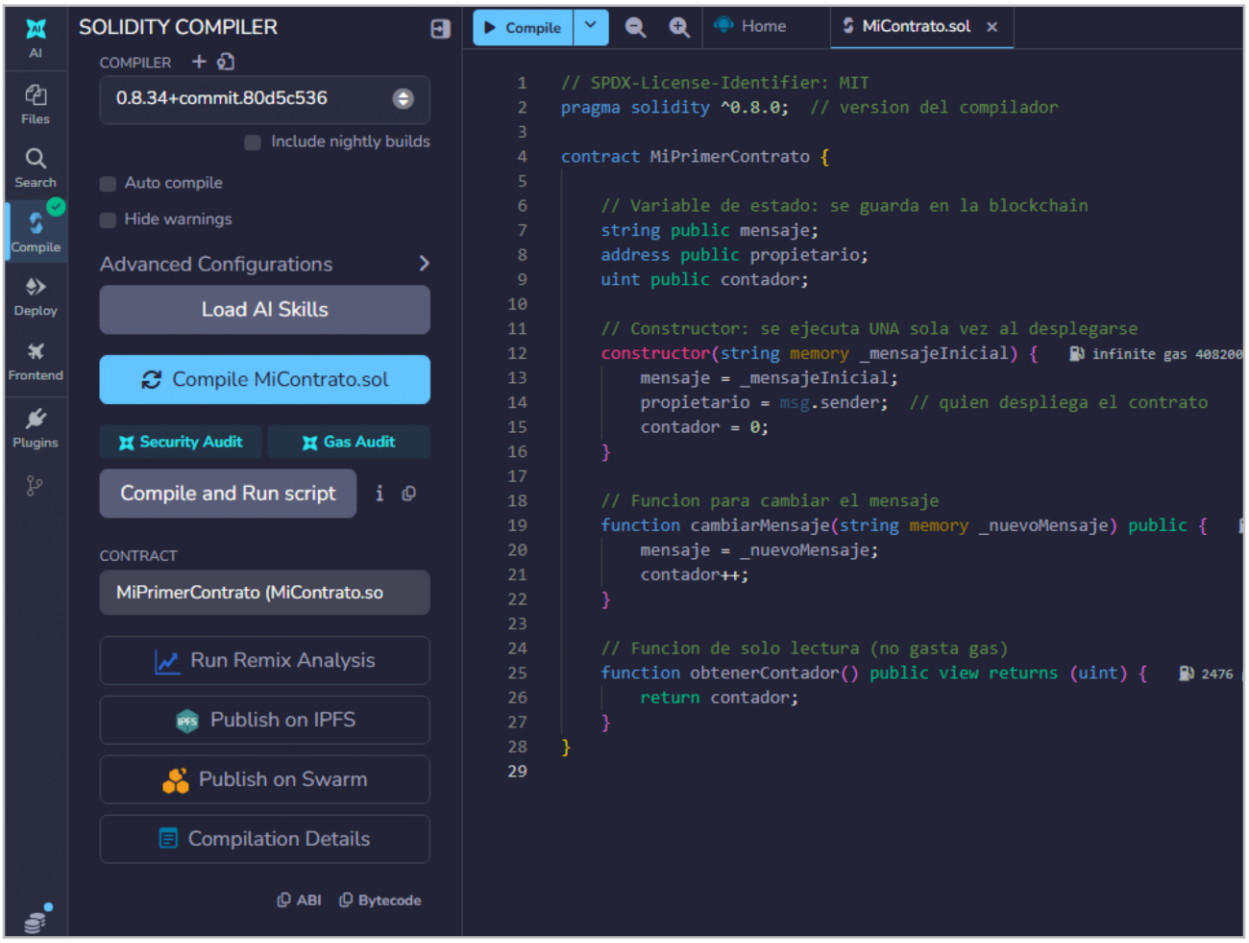
En el explorador de archivos de Remix se creó *MiContrato.sol* dentro de la carpeta *contracts* del espacio de trabajo *default_workspace* y se pegó el contrato de la guía. El código se extrajo por programa del documento de la guía, de modo que el contrato probado es exactamente el que ella entrega (Anexo A). El editor lo mostró sin líneas rojas (Figura 2).

2.4 Paso 3. Compilar el contrato

En el panel del compilador, la versión seleccionada fue la 0.8.34, que cumple la condición de la guía (0.8.0 o superior) y el *pragma ^0.8.0* del contrato. Al pulsar «Compile MiContrato.sol», el icono del compilador mostró la marca verde y el contrato *MiPrimerContrato* quedó disponible para desplegar (Figura 2).

Figura 2

Compilación de MiContrato.sol sin errores



Nota. Captura real de Remix IDE 2.6.6: marca verde en el icono del compilador, versión 0.8.34 y el contrato MiPrimerContrato seleccionado.

2.5 Paso 4. Despliegue y pruebas del contrato original

Se siguió el procedimiento de la guía en Remix VM (Cancun). La Tabla 9 muestra cada acción y su resultado. Remix muestra el resultado de una sola función a la vez, por eso las lecturas se presentan como cuatro capturas (Figura 3), y las transacciones, en la consola (Figura 4).

Tabla 9

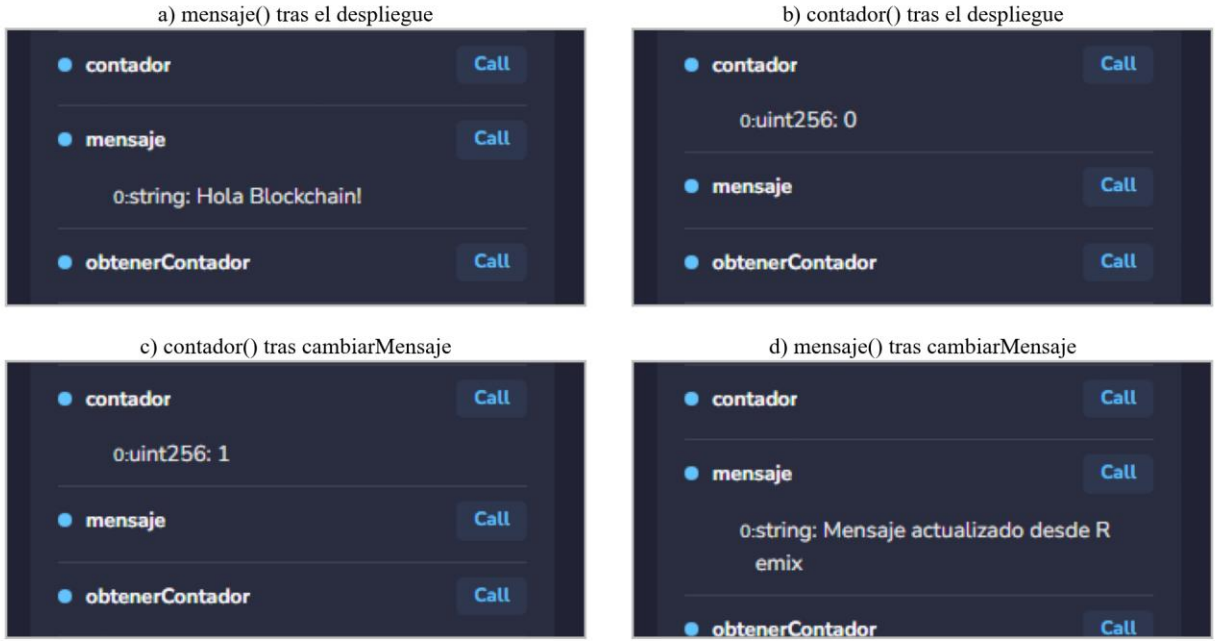
Pruebas del contrato original en Remix VM (Cancun)

Acción	Resultado observado	Evidencia
Constructor con el mensaje «Hola Blockchain!» y Deploy	Transacción de creación exitosa; contrato en 0xd91…39138.	Figura 4a
Botón *mensaje*	0: string: Hola Blockchain!	Figura 3a
Botón *contador*	0: uint256: 0	Figura 3b
***cambiarMensaje* con «Mensaje actualizado desde Remix»**	Transacción exitosa en el bloque 2.	Figura 4b
Botón *contador*	0: uint256: 1	Figura 3c
Botón *mensaje*	0: string: Mensaje actualizado desde Remix	Figura 3d

Nota. Elaboración propia con los resultados mostrados por Remix.

Figura 3

Contrato original funcionando: mensaje y contador

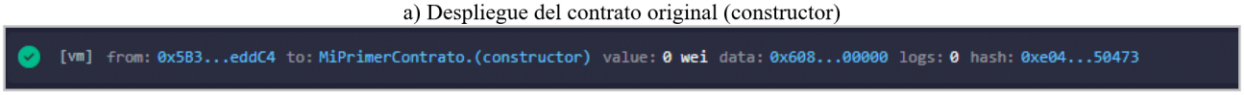


Nota. Recortes reales del panel de contratos desplegados de Remix, en orden cronológico. En d) el texto se parte en dos líneas por el ancho del panel.

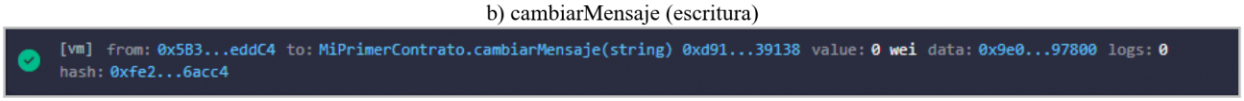
Figura 4

Transacciones registradas en la consola de Remix

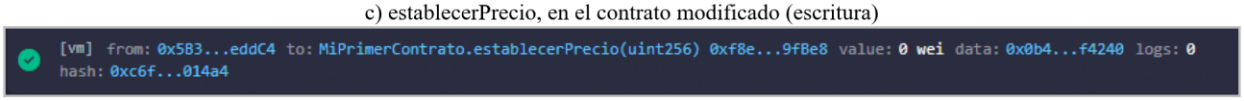
a) Despliegue del contrato original (constructor)



b) cambiarMensaje (escritura)



c) establecerPrecio, en el contrato modificado (escritura)



Nota. Recortes reales de la consola. Cada transacción exitosa lleva la marca verde. La transacción c) corresponde al contrato modificado de la sección 2.6.

El contador pasó de 0 a 1 por una sola llamada a *cambiarMensaje*, y el mensaje cambió: el estado del contrato quedó guardado entre una transacción y la siguiente. Las lecturas, en cambio, no generaron transacciones nuevas: en la consola aparecen como «CALL» (sección 2.7).

2.6 Actividad: agregar precio y establecerPrecio

La actividad pide agregar una variable *uint* llamada *precio* y una función *establecerPrecio(uint _precio)* que la modifique. Se hizo con el mismo patrón que *mensaje* y *cambiarMensaje*: una variable de estado pública, que crea sola su función de lectura, y una función que escribe en ella. Los fragmentos agregados son estos; el contrato completo está en el Anexo B.

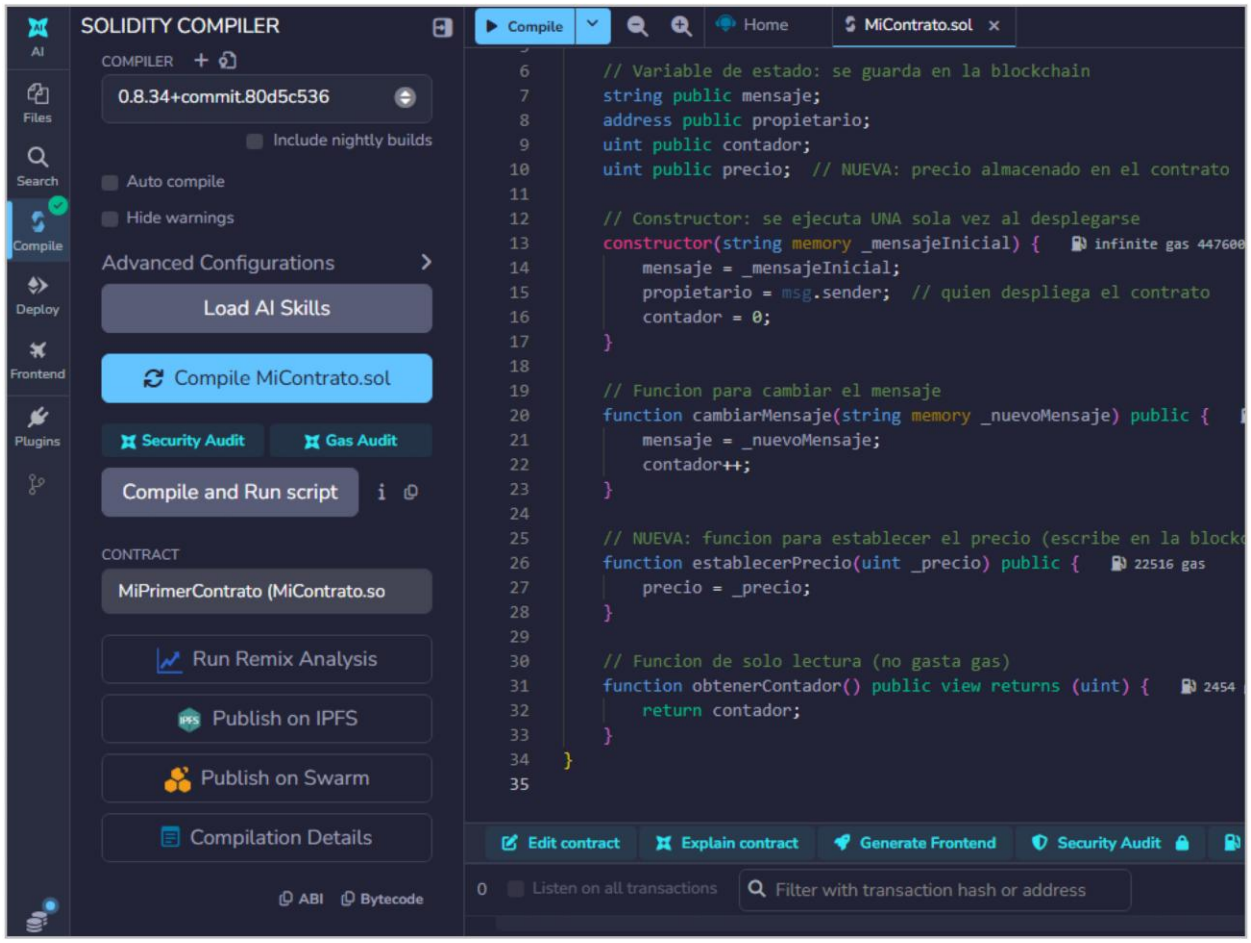
```
uint public precio;  // NUEVA: precio almacenado en el contrato

// NUEVA: funcion para establecer el precio (escribe en la blockchain)
function establecerPrecio(uint _precio) public {
    precio = _precio;
}
```

El contrato se compiló sin errores (Figura 5) y se desplegó como una instancia nueva, en 0xf8e…9fBe8, con el mismo mensaje inicial. Las funciones de la instancia incluyen ahora *establecerPrecio* y *precio*, que la instancia anterior no tenía. Se llamó *establecerPrecio(1000000)*, una transacción exitosa (Figura 4c), y luego *precio*, que devolvió 1000000 (Figura 6).

Figura 5

Contrato modificado compilado sin errores

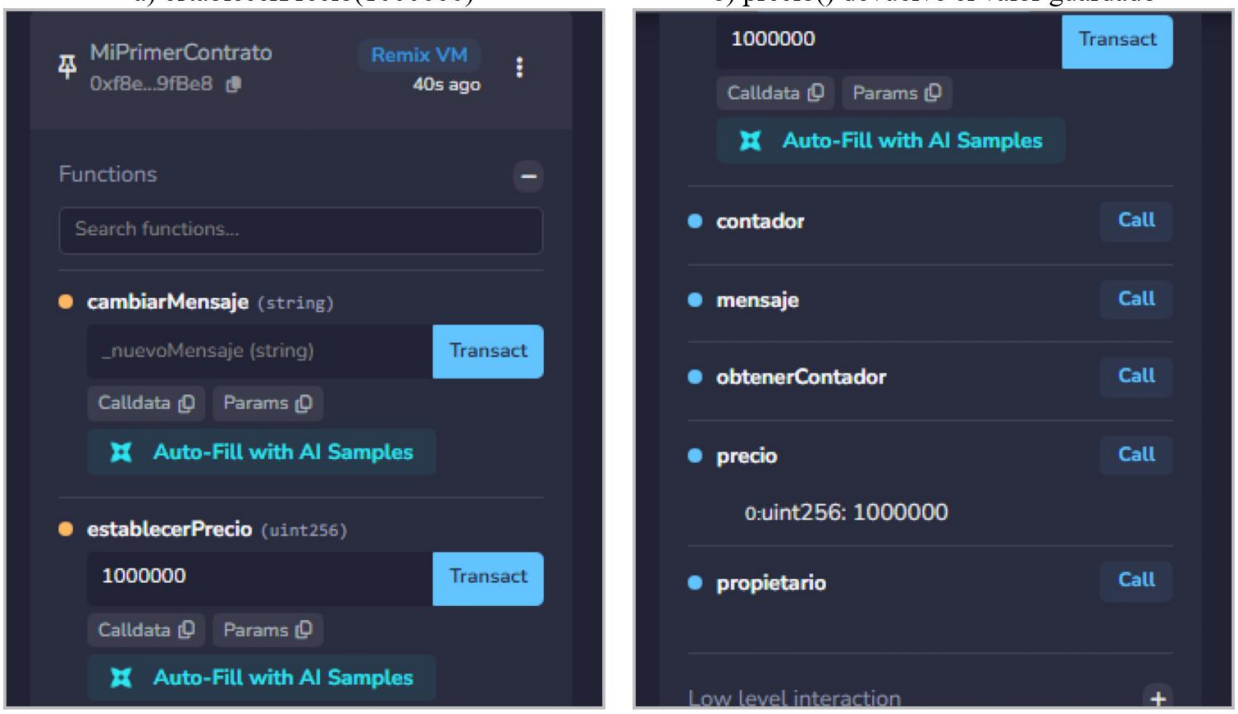


Nota. Captura real de Remix: el editor muestra la variable precio y la función establecerPrecio, y el compilador la marca verde.

Figura 6

Contrato modificado desplegado y funcionando

a) establecerPrecio(1000000) b) precio() devuelve el valor guardado



Nota. Recortes reales del panel de contratos desplegados: la escritura de precio y su lectura.

2.7 Reflexión: por qué escribir cuesta gas y leer no

Pregunta. Cada vez que se llama a *cambiarMensaje* se genera una transacción que cuesta gas. ¿Por qué las operaciones de escritura en la blockchain cuestan dinero pero las de lectura son gratuitas?

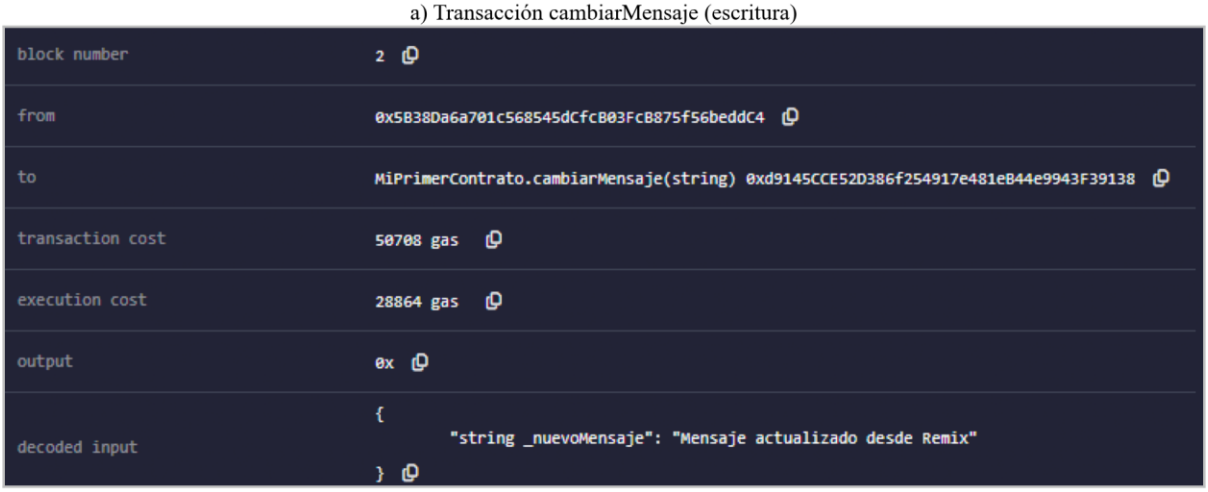
Respuesta. Una escritura cambia el estado compartido de la red, y ese cambio lo deben ejecutar, verificar y guardar todos los nodos; además queda en el almacenamiento, que es la parte más cara de la máquina virtual (Solidity, s. f.-b). El gas pone precio a ese trabajo, evita el spam y los ciclos infinitos, y compensa a quienes validan (ethereum.org, 2026a). Una lectura, en cambio, solo consulta la copia local del estado que ya tiene un nodo: no cambia nada, no necesita consenso y no genera una transacción.

Las mediciones de Remix lo muestran (Figura 7). La transacción *cambiarMensaje* costó 50.708 gas. La llamada *contador()* no creó ninguna transacción: Remix muestra un costo de ejecución de 2.447 gas con la nota «Cost only applies when called by a contract», es decir, ese costo solo se aplicaría si otro contrato la invocara dentro de una transacción. Por eso la gratuidad de la lectura tiene una condición: una función *view* es gratis al consultarla desde fuera, pero no cuando forma parte de una transacción.

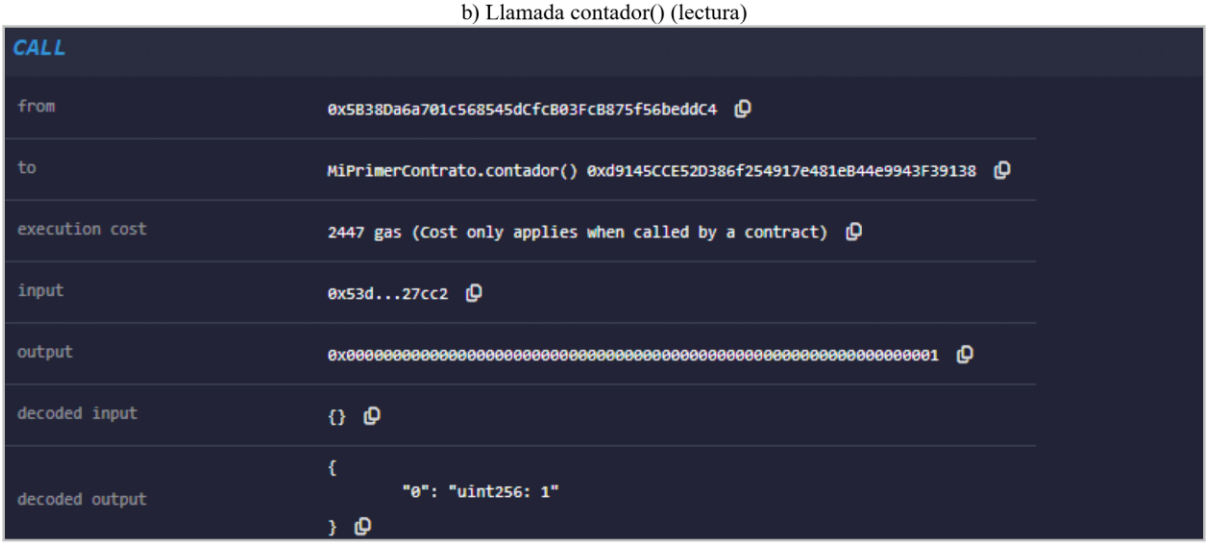
Figura 7

Costo de una escritura frente a una lectura en Remix

a) Transacción cambiarMensaje (escritura)



b) Llamada contador() (lectura)



Nota. Recortes reales del detalle de la consola de Remix: a) la transacción cambiarMensaje y b) la llamada contador().

Los 50.708 gas de la escritura se pueden descomponer y la suma coincide con la medida (Tabla 10).

Tabla 10

Descomposición del costo de la transacción cambiarMensaje

Concepto	Gas	Cómo se obtiene
Costo base de toda transacción	21.000	Mínimo que cuesta una transacción: la transferencia estándar de ETH requiere exactamente 21.000 (ethereum.org, 2026a).
Datos de entrada	844	100 bytes: 37 distintos de cero a 16 gas y 63 en cero a 4 gas (Akhunov et al., 2019).
Ejecución de la función	28.864	*Execution cost* de Remix: incluye escribir *mensaje* y *contador* en el almacenamiento.
Total (*transaction cost*)	50.708	Suma de los tres conceptos; coincide con el valor de Remix.

Nota. Elaboración propia con las cifras de Remix y las reglas de cobro de los datos de entrada. Los 100 bytes son el selector de la función (4), el desplazamiento (32), la longitud (32) y el texto de 31 caracteres relleno a 32 bytes.

La conclusión práctica es que el costo de una escritura depende de cuánto se guarda y cuántas veces se hace. Por eso en los contratos reales, como Esbrinachain, se recomienda evitar variables innecesarias y ciclos largos, y por eso el diseño de la sección 2.2 guarda una huella de 32 bytes por evidencia y no el archivo.

Conclusiones

Un contrato inteligente es código y datos que residen en una dirección de la cadena y que aplican reglas escritas de antemano, sin intermediarios. El contrato de la guía reúne los elementos básicos de Solidity: variables de estado, un constructor que se ejecuta una vez, funciones que escriben y funciones *view* que solo leen.

El contrato compiló sin errores con la versión 0.8.34, se desplegó en Remix VM (Cancun) y se comportó como se esperaba: el mensaje inicial se leyó, el contador pasó de 0 a 1 tras una transacción y el mensaje cambió. La modificación con *precio* y *establecerPrecio* siguió el mismo patrón y devolvió 1.000.000 al leerla.

Las mediciones de Remix respondieron la reflexión con datos: una escritura costó 50.708 gas, de los cuales 21.000 son el mínimo de toda transacción, y una lectura externa no generó transacción alguna. La lectura solo cuesta gas si la invoca otro contrato.

El diseño de *EntregaEvidencias* aplica lo aprendido: roles con modificadores, estados y plazos evaluados al llamar, una huella en vez del archivo y una nota que certifica la entrega, no la calidad. Su límite principal es que un contrato no puede verificar el contenido de una evidencia, por lo que se mantiene la revisión humana.

Referencias

Akhunov, A., Ben Sasson, E., Brand, T., Guthmann, L., y Levy, A. (2019). *EIP-2028: Transaction data gas cost reduction*. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-2028

Blockchain práctico. (s. f.). [Libro; lectura previa del Lab 3, sección 3.1 «Smart Contracts»].

ethereum.org. (2026a, 24 de junio). *Gas and fees*. https://ethereum.org/developers/docs/gas/

ethereum.org. (2026b, 25 de febrero). *Introduction to smart contracts*. https://ethereum.org/developers/docs/smart-contracts/

Remix IDE (Versión 2.6.6) [Software]. (s. f.). https://app.remix.live

Solidity. (s. f.-a). *Contracts*. Solidity Documentation. https://docs.soliditylang.org/en/latest/contracts.html

Solidity. (s. f.-b). *Introduction to smart contracts*. Solidity Documentation. https://docs.soliditylang.org/en/latest/introduction-to-smart-contracts.html

Solidity. (s. f.-c). *Types*. Solidity Documentation. https://docs.soliditylang.org/en/latest/types.html

Universidad Manuela Beltrán. (2026). *Guía de laboratorio n.º 3: Solidity y contratos* [Guía de laboratorio, módulo 1: Aplicaciones multiplataforma, Nuevas Tecnologías del Desarrollo]. UMB.

Anexo A. Contrato original (MiContrato.sol)

Contrato de la guía, extraído por programa del documento de la guía y probado en Remix.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;  // version del compilador

contract MiPrimerContrato {

    // Variable de estado: se guarda en la blockchain
    string public mensaje;
    address public propietario;
    uint public contador;

    // Constructor: se ejecuta UNA sola vez al desplegarse
    constructor(string memory _mensajeInicial) {
        mensaje = _mensajeInicial;
        propietario = msg.sender;  // quien despliega el contrato
        contador = 0;
    }

    // Funcion para cambiar el mensaje
    function cambiarMensaje(string memory _nuevoMensaje) public {
        mensaje = _nuevoMensaje;
        contador++;
    }

    // Funcion de solo lectura (no gasta gas)
    function obtenerContador() public view returns (uint) {
        return contador;
    }
}
```

Anexo B. Contrato modificado (MiContrato_v2.sol)

Contrato con la variable *precio* y la función *establecerPrecio*, probado en Remix.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;  // version del compilador

contract MiPrimerContrato {

    // Variable de estado: se guarda en la blockchain
    string public mensaje;
    address public propietario;
    uint public contador;
    uint public precio;  // NUEVA: precio almacenado en el contrato

    // Constructor: se ejecuta UNA sola vez al desplegarse
    constructor(string memory _mensajeInicial) {
        mensaje = _mensajeInicial;
        propietario = msg.sender;  // quien despliega el contrato
        contador = 0;
    }

    // Funcion para cambiar el mensaje
    function cambiarMensaje(string memory _nuevoMensaje) public {
        mensaje = _nuevoMensaje;
        contador++;
    }

    // NUEVA: funcion para establecer el precio (escribe en la blockchain)
    function establecerPrecio(uint _precio) public {
        precio = _precio;
    }

    // Funcion de solo lectura (no gasta gas)
    function obtenerContador() public view returns (uint) {
        return contador;
    }
}
```